Practical 3: API Integration and Data Rendering in React	
Objective	To consume a REST API in React and handle asynchronous data with loading and error states.
Architecture / Diagram	
Diagram	Projects.jsx ├── useEffect() → triggers fetch on mount ├── useState: data, loading, error ├── [loading] → <Spinner /> ├── [error] → <ErrorMessage /> └── [success] → <RepoList data={data} />
Problem Definition	
Problem Statement	• Integrate a public REST API (e.g., GitHub API) into the portfolio application to fetch and display a list of repositories dynamically. • Use the Projects page built in Practical 2 as the integration point. • Implement a loading spinner component shown while the request is in progress. • Implement an error message component shown if the API call fails. • Render at least the repository name and URL for each item returned.

Steps	1. Inside Projects.jsx, set up 3 state variables for data, loading, and error:
	const [repos, setRepos] = useState([]);
	const [loading, setLoading] = useState(true);
	const [error, setError] = useState(null);
	
	`import { useState, useEffect } from 'react'`
	
	`function Projects({ darkMode }) {`
	`  const [repos, setRepos] = useState([])`
	`  const [loading, setLoading] = useState(true)`
	`  const [error, setError] = useState(null)`
	`  const [searchTerm, setSearchTerm] = useState('')`

2. Use useEffect to fetch data when the component mounts:

```
useEffect(() => {
  fetch('https://api.github.com/users/<username>/repos')
    .then((res) => res.json())
    .then((data) => setRepos(data))
    .catch((err) => setError(err.message))
    .finally(() => setLoading(false));
}, []);
```

student-portfolio > src > pages > Projects.jsx > Projects

```jsx
import { useState, useEffect } from 'react'

function Projects({ darkMode }) {
  const [repos, setRepos] = useState([])
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState(null)
  const [searchTerm, setSearchTerm] = useState('')

  useEffect(() => {
    const fetchRepos = async () => {
      try {
        setLoading(true)
        setError(null)
        const response = await fetch('https://api.github.com/users/katariyajignesh266/repos')
        if (!response.ok) throw new Error('Failed to fetch repositories')
        const data = await response.json()
        setRepos(data)
      } catch (err) {
        setError(err.message)
      } finally {
        setLoading(false)
      }
    }
    fetchRepos()
  }, [])
```

3. Conditionally render based on state:

```
if (loading) return <Spinner />;
if (error) return <ErrorMessage message={error} />;
```

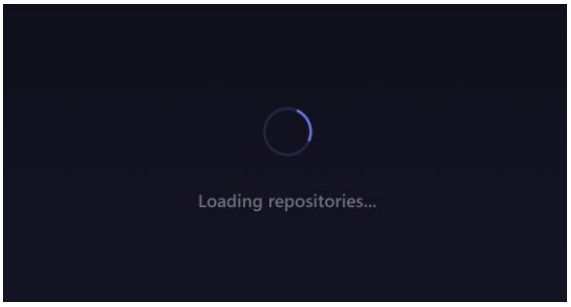
```jsx
const LoadingSpinner = () => (
  <div style={loadingContainerStyle}>
    <div style={spinnerStyle}></div>
    <p style={loadingTextStyle}>Loading repositories...</p>
  </div>
)
```

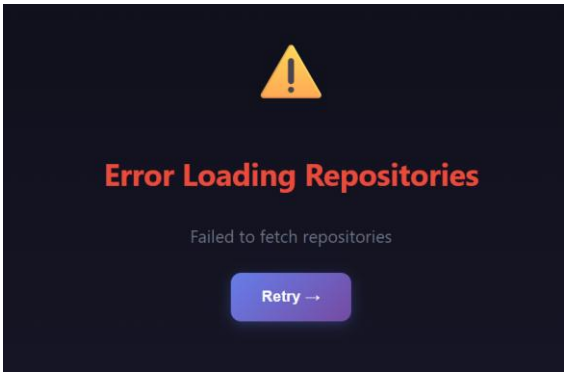
4. Map over the repos array and render name and html_url for each repository.

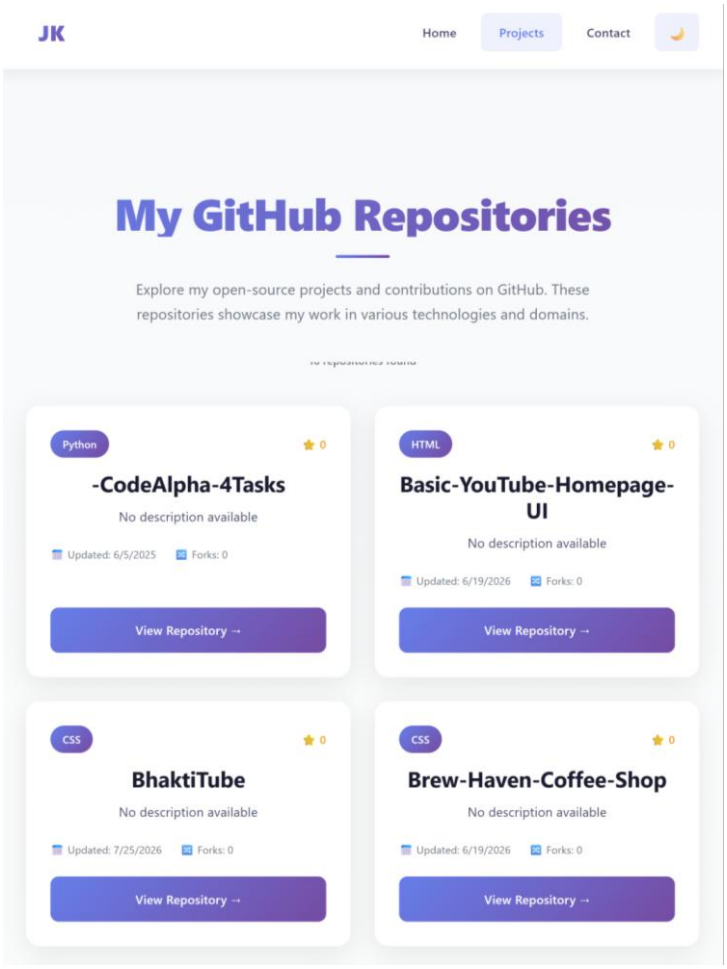
```
const RepositoryCard = ({ repo }) => (
  <div style={{
    ...repoCardStyle,
    background: darkMode ? '#1a1a2e' : 'white',
    boxShadow: darkMode ? '0 10px 40px rgba(0, 0, 0, 0.3)' : '0 10px 40px rgba(0, 0, 0, 0.08)'
  }}>
    <div style={repoCardHeaderStyle}>
      <span style={tagStyle}>{repo.language || 'Unknown'}</span>
      <div style={starStyle}>
        ⭐ {repo.stargazers_count}
      </div>
    </div>
    <h2 style={{
      ...projectTitleStyle,
      color: darkMode ? '#f0f0f0' : '#1a1a2e'
    }}>{repo.name}</h2>
    <p style={{
      ...descriptionStyle,
      color: darkMode ? '#b0b0c0' : '#4a4a6a'
    }}>
      {repo.description || 'No description available'}
    </p>
    <div style={repoMetaStyle}>
      <span style={metaItemStyle}>
        🗓️ Updated: {new Date(repo.updated_at).toLocaleDateString()}
      </span>
      <span style={metaItemStyle}>
        🔀 Forks: {repo.forks_count}
      </span>
    </div>
    <a
      href={repo.html_url}
      target="_blank"
      rel="noopener noreferrer"
      style={linkStyle}
    >
      View Repository →
    </a>
  </div>
)
```

5. Test the happy path confirm data renders correctly after loading.
(Already shown in Output)
6. Test the error path temporarily break the API URL and confirm the error message displays.
(Already shown in Output)

Output





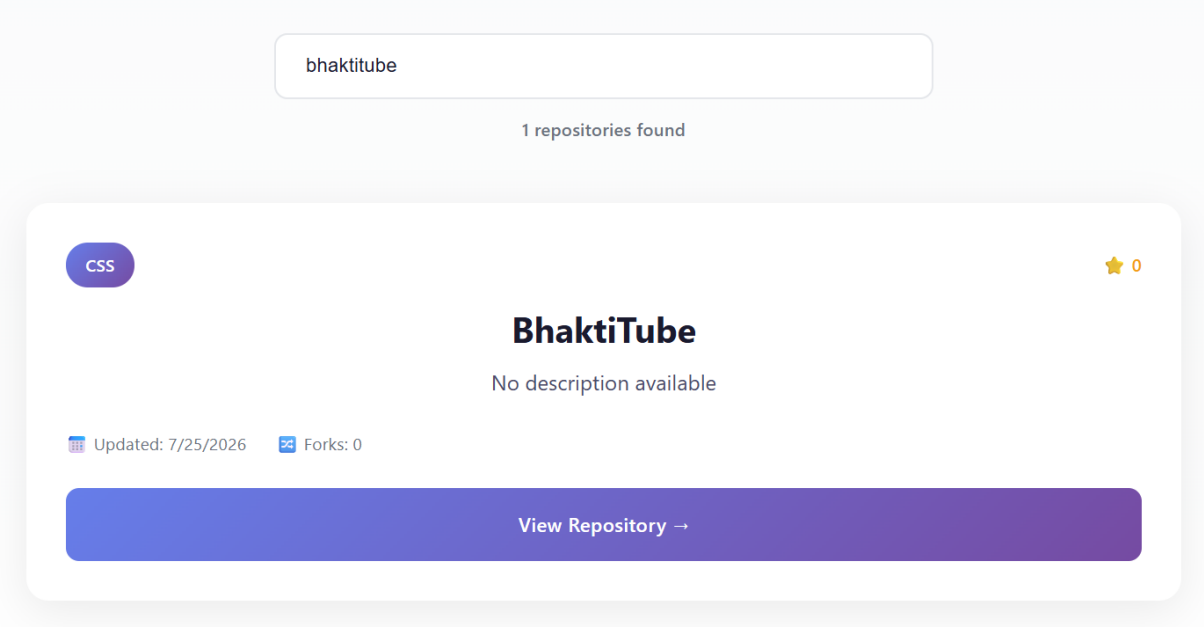


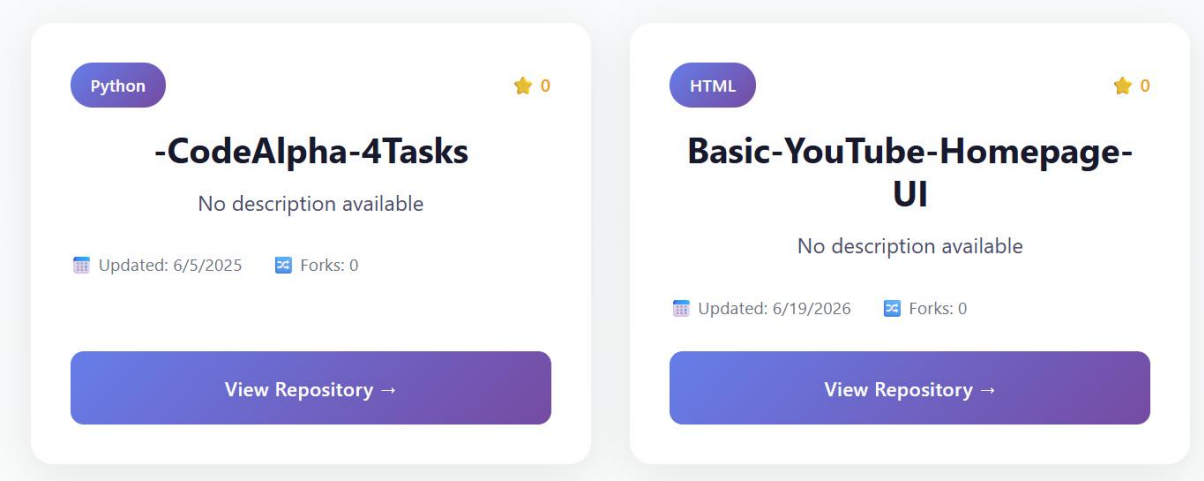
Key Questions / Analysis

1.Why is useEffect required to trigger a fetch on component mount instead of calling fetch directly in the component body?

useEffect runs after the component renders, ensuring the API is called only when needed. Calling fetch directly in the component body would trigger it on every render, causing repeated API requests.

2.What is the difference between a loading state and an error state, and why must both be handled separately?

	A loading state indicates data is still being fetched, while an error state indicates the request failed. Handling both separately provides appropriate feedback for each situation. **3.How would the user experience change if loading and error states were not implemented?** Users might see a blank screen, outdated data, or no explanation when a request fails, making the application confusing and less reliable.
Supplementary Problems	• **Add a retry button that re-triggers the fetch when an error occurs.**  • **Add a search input that filters the rendered repository list by name.**  • **Display the repository's star count alongside its name.**

	
Key Skills Addressed	• Fetching data from a REST API using fetch or axios • Using useEffect to trigger side effects on component mount • Managing asynchronous state loading, success, and error • Conditionally rendering UI based on request state
Learning Outcome	Students will be able to consume a REST API in React, manage asynchronous state correctly, and provide appropriate UI feedback during all stages of a network request.